

Guide d'édition et de déploiement du site.

Comment travailler sur **virtuslever.com** — la stack, les fichiers à connaître, et la procédure complète pour publier une nouvelle version.

SITE

virtuslever.com

HÉBERGEMENT

Cloudflare Pages

SOURCE

github.com/CalilDrissi/lever

PUBLIC

Équipe Virtus Lever

Ce que c'est, où ça vit.

Le site **virtuslever.com** est une landing page statique générée par Next.js. Chaque commit poussé sur la branche `main` du dépôt déclenche un build et un déploiement automatique vers Cloudflare Pages.

Adresses utiles

Production	https://virtuslever.com
Page de style	https://virtuslever.com/styleguide/ — référence visuelle des couleurs, typo et composants.
Dépôt GitHub	github.com/CalilDrissi/lever
Dashboard Cloudflare	Workers & Pages → projet virtus-lever

À RETENIR Les URLs `*.pages.dev` sont volontairement bloquées : un redirect 301 renvoie toujours vers `virtuslever.com` afin d'éviter le contenu dupliqué côté SEO. Pas la peine de les partager, elles n'affichent plus le site.

Les outils qui font tourner le site.

Site statique généré au build : aucun serveur Node ne tourne en production.
Tout est précompilé en HTML/CSS/JS et servi depuis le CDN Cloudflare.

Next.js 14 · App Router

Framework React. Configuré en `output: "export"` pour générer un site 100% statique.

TypeScript

Typage strict. `npm run typecheck` garde la base saine.

Tailwind CSS 3

Système de tokens dans `tailwind.config.ts`. La palette est calquée sur celle de Superhuman (neutral, purple, mulberry, green, ...).

framer-motion

Toutes les animations (parallax, fade-up, marquee, score gauge). Respecte `prefers-reduced-motion`.

lucide-react

Bibliothèque d'icônes. Pas d'emoji dans l'UI.

Cloudflare Pages

Hébergement, CDN, certificat SSL. Une fonction edge (`functions/_middleware.js`) gère les redirects canoniques.

Node 20

Version cible pour les builds (locaux et CI).

Où trouver quoi.

Les dossiers à connaître. La grande majorité des modifications de contenu se passent dans `lib/copy.ts` ; les ajustements visuels dans `tailwind.config.ts` et les fichiers de `components/sections/`.

CHEMIN	RÔLE
<code>app/</code>	Pages Next.js. <code>page.tsx</code> compose toutes les sections, <code>layout.tsx</code> contient le <code><head></code> et les meta tags SEO.
<code>app/styleguide/</code>	Page <code>/styleguide</code> — la référence vivante de tous les tokens et composants.
<code>components/sections/</code>	Une section par fichier (<code>hero.tsx</code> , <code>pricing.tsx</code> , <code>faq.tsx</code> , etc.). Chacun est self-contained.
<code>components/ui/</code>	Composants atomiques réutilisables (<code>Button</code> , <code>Badge</code> , <code>Card</code> , <code>Input</code>).
<code>components/illustrations/</code>	Illustrations SVG inline (les 4 vignettes des piliers).
<code>lib/copy.ts</code>	Toute la copie du site , en FR et EN. Source unique de vérité — éditer ici plutôt que dans les composants.
<code>lib/images.ts</code>	Référence centralisée des photos Pexels utilisées dans les sections.
<code>lib/asset.ts</code>	Helper <code>asset()</code> pour les chemins d'asset manuels. À utiliser pour toute référence vers <code>/public/</code> qui n'est pas un <code>next/image</code> ou <code>next/link</code> .
<code>public/</code>	Assets servis tels quels (logos PNG des fournisseurs, vidéo hero, favicons).
<code>functions/</code>	Edge functions Cloudflare. <code>_middleware.js</code> renvoie un 301 de <code>*.pages.dev</code> vers <code>virtuslever.com</code> .
<code>tailwind.config.ts</code>	Couleurs, typographie (clamp fluide), spacing, radius, shadows.
<code>next.config.mjs</code>	Configuration de build (export statique, images, basePath conditionnel).

Faire tourner le site sur ta machine.

Prérequis : **Node 20+** et **npm**. Sur Mac : `brew install node`. Sur Windows : installer depuis nodejs.org.

Cloner et installer

```
git clone https://github.com/CalilDrissi/lever.git
cd lever
npm install
```

Lancer le serveur de développement

```
npm run dev
```

Le site est disponible sur `http://localhost:3000`. Les modifications sont reflétées en direct (Fast Refresh).

Commandes utiles

COMMANDE	EFFET
<code>npm run dev</code>	Lance le serveur de dev sur le port 3000.
<code>npm run build</code>	Génère le site statique dans <code>out/</code> .
<code>npm run typecheck</code>	Vérifie les types TypeScript (sans build).
<code>npm run lint</code>	Lance l'ESLint Next.js.

ASTUCE Toujours lancer `npm run typecheck` avant de pousser un commit. Une erreur de type empêche le déploiement.

Changer un texte, un tarif, une FAQ.

Toute la copie du site vit dans `lib/copy.ts`. Pour éditer un titre, un prix ou une question, il suffit d'ouvrir ce fichier — pas besoin d'aller chercher dans les composants.

Structure du fichier

Le fichier exporte un objet `copy` avec deux clés : `fr` (français — utilisé par le site) et `en` (anglais — miroir prêt pour une future internationalisation, pas encore branché à l'UI).

Exemples

Changer le titre du hero

```
// lib/copy.ts → copy.fr.hero.tagline
hero: {
  tagline: "Transforme ton inbox en 1 priorité claire par jour.",
  ...
}
```

Modifier les tarifs

```
// lib/copy.ts → copy.fr.pricing.plans
plans: [
  {
    key: "lever",
    name: "Lever",
    monthlyPrice: 18, // ← change ici
    annualPrice: 15, // ← et ici
    ...
  },
  ...
]
```

Ajouter une FAQ

```
// lib/copy.ts → copy.fr.faq.items
items: [
  { q: "...", a: "..." },
  { q: "Nouvelle question ?", a: "Réponse." }, // ← ajouter
]
```

Règles de ton

- **FR primaire**, tutoyer.
- **Phrases courtes**, directes. Pas de "révolutionnaire", "magique", "10x", "découvrez".
- **Minuscules pour les unités** ("10 min/jour", "–30 % de backlog").
- **Pas d'emoji** dans les composants — uniquement des icônes lucide.

Couleurs, typographie, espacements.

Le système de design est centralisé dans `tailwind.config.ts`. Modifier un token ici met à jour l'ensemble du site. Voir la page `/styleguide` pour visualiser chaque token.

Palette de couleurs

Sept échelles, calquées sur la design system de Superhuman :

neutral	Fond, type, bordures. <code>neutral-90</code> = encre, <code>neutral-5</code> = surface claire.
purple	Accent de marque. <code>purple-60</code> = couleur d'action principale (CTA, focus).
mulberry	Tons sombres profonds (footer, wordmark fantôme).
green	Indicateurs positifs (scores, "traité"). <code>green-60</code> = signal.
red / blue / sky / orange	Échelles secondaires pour erreurs, highlights, etc.

Typographie

Toutes les tailles à partir de `h6` utilisent `clamp()` pour scaler fluidiquement entre mobile et desktop. Les valeurs sont définies dans `tailwind.config.ts` sous `fontSize`.

Remplacer la vidéo du hero

La vidéo de fond du hero est dans `public/hero-bg.mp4`. Pour la remplacer :

1. Optimiser le fichier (H.264, 1080p max, < 3MB idéalement).
2. Le déposer dans `public/` sous le même nom.
3. Optionnel : ajouter `hero-bg.webm` (version VP9) et `hero-bg.jpg` (poster).

Changer une image Pexels

Les IDs Pexels utilisés sont listés dans `lib/images.ts`. Trouver une nouvelle photo sur pexels.com, noter l'ID dans l'URL, et le remplacer dans la définition.

ATTENTION Les chemins `/asset` manuels (vidéo, poster, logos) doivent passer par `asset("/chemin")` pour fonctionner en production. `next/image` et `next/link` gèrent ça automatiquement.

Publier une nouvelle version.

Le site est hébergé sur **Cloudflare Pages**. Le déploiement se fait **manuellement** depuis ta machine avec l'outil en ligne de commande `wrangler`. Un `git push` seul ne suffit pas — il faut explicitement déclencher la publication.

POURQUOI ? Ce choix garde le déploiement sous contrôle humain. Pas de surprise, pas de build qui part en production sans qu'on l'ait validé. En contrepartie, il y a une étape de plus à exécuter après chaque changement.

Pré-requis — une seule fois par personne

1. Token API Cloudflare

Chaque personne qui déploie a besoin de son propre token. Pour en créer un :

1. Ouvrir dash.cloudflare.com/profile/api-tokens.
2. Cliquer **Create Token** → en bas, **Custom token** → **Get started**.
3. Nommer le token, par exemple `virtus-lever-deploy-prenom`.
4. Sous **Permissions**, ajouter une ligne :
Account → Cloudflare Pages → Edit.
5. Sous **Account Resources** : Include → ton compte.
6. Optionnel : définir une date d'expiration.
7. Valider → copier le token affiché (visible une seule fois).

2. Stocker le token localement

Le plus simple : créer un fichier `.env.local` à la racine du projet (déjà dans le `.gitignore`, donc jamais commité) :

```
# .env.local
CLOUDFLARE_API_TOKEN="ton_token_ici"
CLOUDFLARE_ACCOUNT_ID="2e6317da2d1dfc46378b391151b150ce"
```

Puis charger ces variables avant de déployer :

```
export $(cat .env.local | xargs)
```

SÉCURITÉ Ne jamais commiter le token dans Git. Ne jamais le coller dans une issue GitHub ou un message Slack. En cas de fuite, le révoquer immédiatement depuis le dashboard API Tokens et en créer un nouveau.

Procédure de déploiement

1 Créer une branche pour ton changement

```
git checkout -b feat/nouveau-titre-hero
```

2 Faire ta modification et tester en local

```
npm run dev  
# vérifier le résultat sur http://localhost:3000
```

3 Vérifier que le code compile

```
npm run typecheck  
npm run build
```

Si l'un des deux échoue, corrige avant de continuer — le déploiement échouera aussi.

4 Committer et pousser sur GitHub

```
git add .  
git commit -m "feat: nouveau titre du hero"  
git push origin feat/nouveau-titre-hero
```

Ouvrir une Pull Request, faire relire, merger sur `main` quand tout est validé.

5

Récupérer la dernière version de `main`

```
git checkout main
git pull origin main
```

6

Construire le site

```
npm run build
```

La sortie est générée dans `out/`.

7

Déployer sur Cloudflare Pages

```
# Si tes variables sont déjà chargées via .env.local
npx wrangler pages deploy out \
  --project-name=virtus-lever \
  --branch=main

# Sinon, les passer en ligne :
CLOUDFLARE_API_TOKEN="ton_token" \
CLOUDFLARE_ACCOUNT_ID="2e6317da2d1dfc46378b391151b150ce" \
npx wrangler pages deploy out \
  --project-name=virtus-lever \
  --branch=main
```

La commande affiche une URL `*.pages.dev` de preview en fin d'exécution. Le déploiement est terminé en ~30 secondes.

8

Vérifier en production

Ouvrir virtuslever.com et contrôler ton changement. Si quelque chose cloche, voir la section dépannage.

Déploiement de test (preview)

Pour publier sur une URL temporaire sans toucher à la production, utiliser une branche autre que `main` :

```
npx wrangler pages deploy out \  
  --project-name=virtus-lever \  
  --branch=preview-mon-changement
```

La sortie inclut une URL `preview-mon-changement.virtus-lever.pages.dev`. Attention : ces URLs sont redirigées vers `virtuslever.com` par défaut (cf. section 03). Pour les rendre accessibles, désactiver temporairement le redirect dans `functions/_middleware.js`.

RACCOURCI Pour gagner du temps, créer un alias dans ton shell :

```
alias lever-deploy='npm run build && npx wrangler pages deploy out --project-  
name=virtus-lever --branch=main'
```

Une fois `.env.local` chargé, taper `lever-deploy` suffit.

La checklist.

Trois minutes de vérification évitent un site cassé en production. À faire systématiquement avant chaque PR.

Code

- `npm run typecheck` — aucun message d'erreur.
- `npm run build` — termine avec le rapport "Static / prerendered as static content".
- `npm run lint` — pas d'erreur (les warnings peuvent passer si justifiés).

Visuel

- Tester sur mobile (responsive). Le site est conçu pour rester lisible jusqu'à 360 px de large.
- Vérifier que la nouvelle section ne casse pas la séquence des sections claires/sombres.
- Si tu as ajouté une image, vérifier qu'elle charge en production (et pas seulement en local).

Copie

- Relire en français — tutoyer, phrases courtes, pas de fautes.
- Si tu as utilisé une marque ou un nom propre (ex. Gmail, Apple), vérifier que c'est justifié et pas une promesse marketing fausse.
- Ajouter aussi la version anglaise dans `copy.en` quand c'est possible (pour la future i18n).

Quand ça coince.

Les problèmes les plus fréquents et leur solution.

Le build échoue avec une erreur TypeScript

Lancer `npm run typecheck` en local pour voir le détail. Souvent : un import manquant, un type qui n'existe pas dans `copy.ts`, ou un composant qui a changé de signature.

L'image apparaît en local mais pas en production

Le chemin n'utilise pas `asset()`. Tout chemin vers `/public/` dans un `<img>`, `<source>` ou `style={{ backgroundImage }}` doit passer par le helper :

```
import { asset } from "@lib/asset";

<img src={asset("/mon-image.png")} alt="..." />
```

Le site sur `virtus-lever.pages.dev` redirige

C'est voulu. La fonction `functions/_middleware.js` renvoie un 301 vers `virtuslever.com` pour éviter le contenu dupliqué côté SEO. Pour tester un preview deploy, regarder les liens fournis dans la PR sur GitHub.

`wrangler pages deploy` renvoie une erreur d'authentification

Le token ou l'account ID sont absents ou incorrects. Vérifier :

- Que `.env.local` est bien chargé : `echo $CLOUDFLARE_API_TOKEN` doit renvoyer le token.
- Que le token n'a pas expiré (dashboard → API Tokens).
- Que la permission est bien Account · Cloudflare Pages · Edit.

La nouvelle version n'apparaît pas sur `virtuslever.com`

Trois causes possibles, à vérifier dans l'ordre :

1. **Le déploiement n'a pas tourné.** Un `git push` seul ne déclenche rien. Il faut explicitement lancer `npx wrangler pages deploy out`.
2. **Le build d'avant a été poussé.** Refaire `npm run build` avant de redéployer.
3. **Cache navigateur.** Ouvrir la page en navigation privée, ou hard-refresh (`Cmd+Shift+R`). Cloudflare cache aussi côté edge — attendre ~1 min ou purger le cache dans le dashboard.

Suivre l'état des déploiements

Dashboard Cloudflare → Workers & Pages → virtus-lever → onglet Deployments. Chaque `wrangler pages deploy` y apparaît avec son URL de preview, son timestamp et le statut (succès / échec).

Une animation bug sur certains navigateurs

framer-motion + SVG : ne pas se reposer sur `transform-origin` + `transform-box: view-box` pour les rotations avec un point d'ancrage fixe (clock hands, dials). Animer directement les coordonnées via `useTransform` d'un `useMotionValue` (voir `components/illustrations/pillars.tsx` → `FollowupsIllustration` pour l'exemple).

Besoin d'aide ? Pour toute question sur la stack ou une décision de design, ouvrir une issue sur le dépôt GitHub ou contacter Cal directement. Le fichier `CLAUDE.md` (s'il existe) contient les règles de design system détaillées.